点群AIによる茶の新芽の選択摘採ロボット

Physical AI応用1 最終課題
説明資料（数値はすべてシミュレーションの結果で、実機・実茶樹での結果ではない）

1. 課題の概要と目的

茶の選択摘採では、摘む若い新芽と、残す成葉・芽・枝を見分け、枝を傷付けずに1本ずつ摘み取る必要がある。本課題では、ビニールハウスの天井レールを走る台車から吊った台に小型ロボットアーム（Adept ADA031）を載せ、ハウスの骨組み（アーチ）に付けたカメラの点群から、言葉で指定した新芽をAIが特定し、摘んで回収箱へ入れるまでを自動で行うシステムを、物理シミュレーション（MuJoCo）と写実的なレンダリング（Unreal Engine）で構築・評価した。



図1 実寸のパイプハウスと茶樹（Unreal Engine 5.7、Lumen）。茶樹・ハウス・地面は写真を参考にプログラムで生成した。

2. タスク設定と成功条件

ハウスは実運用想定の幅5.7m×長さ10m×高さ3.1m（アーチ12本）、茶樹は高さ1.5〜2.1m。台のまわりの樹冠から新芽が3本出ており、指示文は「新芽を1本摘んで、かごに入れて」「左の…」「真ん中の…」「右の…」の4種類。指定した新芽（任意の場合はどれか1本だけ）を摘み、かごの中へ入れ、他の新芽は株に残し、グリッパーを開いていれば成功。新芽は株に拘束されており、指で握って上へ引いたときだけ外れる（切断・損傷は未モデル化）。

3. システム構成と処理の流れ

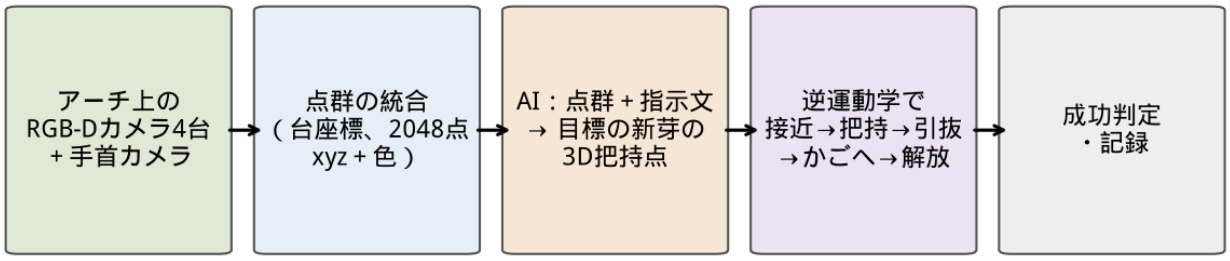


図2 処理の流れ。MuJoCoで計算した全物体・カメラの姿勢をUnrealへ毎ステップ送り、RGBと深度を描画する。



図3 アーチ上の4台のカメラ（上段3枚と下段左）、手首カメラ（下段中央）、統合した点群を上から見た図とAIの推定把持点（下段右、赤丸）。

4. AIを活用した箇所と、その必要性

新芽の位置は株ごと・試行ごとに変わり、茶樹の葉や枝と同じ緑色で重なって見える。決まった座標を巡回するだけでは指定の新芽を摘めないため、カメラの点群と指示文から「どの新芽を、どこで掴むか」をAIが判断する。

方式	入力	出力	役割
新芽位置推定＋IK	点群、指示文	指定新芽の3D把持点	AIが摘採候補を特定し座標を求める。動作は逆運動学
点群VLA	点群、指示文	16ステップ分の関節目標	AIが動作まで直接出す（flow matching）
SmolVLA（比較）	画像2枚、指示文、関節	関節目標のチャンク	事前学習済みVLAの微調整

点群エンコーダは最遠点サンプリングで128中心・各32近傍の局所PointNet＋中心座標のFourier埋め込み。指示文はSmolVLAの言語モデル（SmolVLM2）を凍結して特徴化。学習データはAdeeptの逆運動学によるお手本の成功エピソードのみ（Unreal 200話、MuJoCo 400話）。

5. 結果

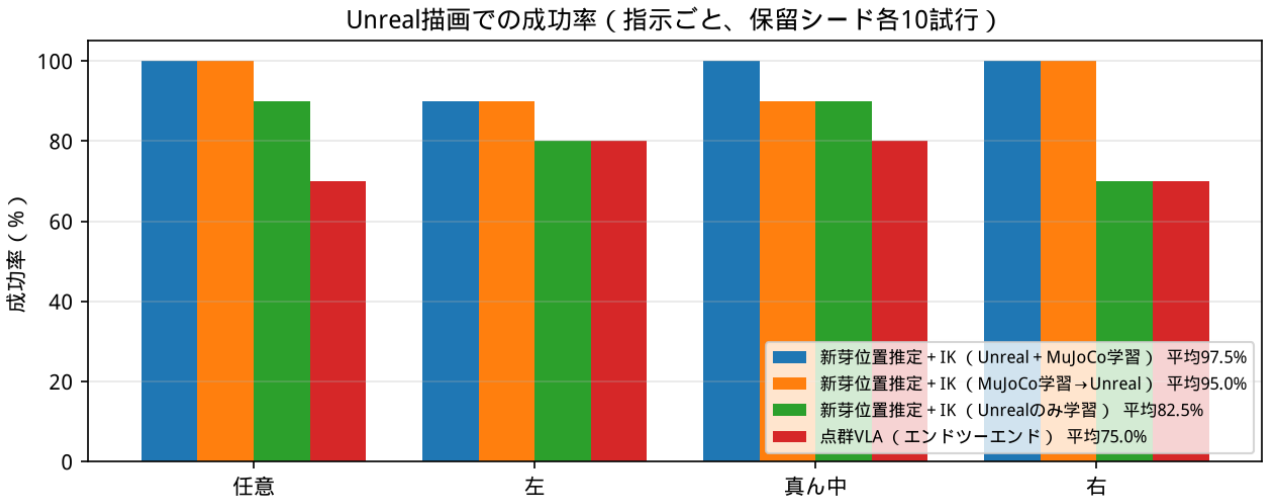


図4 Unreal描画での成功率。新芽位置推定＋IK（Unreal＋MuJoCo学習）が平均97.5%（推定誤差の中央値1.5〜2.5mm）、点群VLAは75%。

MuJoCo描画（茶樹の簡素版）では、新芽位置推定＋IKが97.5%、点群VLA（関節状態なし）75%、点群VLA（関節状態あり）15%、SmolVLA
5%。簡素なMuJoCo画面で学習した位置推定AIは、写実的なUnreal画面でもそのまま95%で動いた。



図5 動画の一場面（左の新芽を摘んでかごへ入れたところ）。全景、アーチ上のカメラ、手首カメラ、点群とAIの推定把持点（赤丸）。

6. 工夫した点と、前の版から発展させた点

・前の版（同じリポジトリの main ブランチ、v7）は、天井吊りの2関節アームと球殻グリッパーで、決まった位置の葉を順に回収するもので、AIは使っていなかった。本版では、言葉で指定した新芽をAIがカメラの点群から探して摘む形に発展させた。

・物理はMuJoCo、描画はUnreal Engineで分担し、MuJoCoで計算した全物体とカメラの姿勢を毎ステップUnrealへ送って、画面のないGPUサーバーで写実的なRGB画像と深度を得る仕組みを作った。UnrealRoboticsLab（MuJoCoとUnrealの連携）の考え方を参考にしたが、そのままでは画面なし環境でカメラ画像が出ず、複数起動で停止したため、姿勢を送る方式に置き換えた。

・茶樹・パイプハウス・地面は、実際の茶樹の写真を参考にプログラムで生成した。外部の3Dアセットは使っていない。

・AIで「どの新芽か・どこか」を推定し、動作は運動学で計画する役割分担型と、動作までAIが出すエンドツーエンド型（点群VLA、SmolVLA）を、同じ条件で比べた。

・簡素なMuJoCo描画で学習したAIを、写実的なUnreal描画にそのまま使えるか確かめ（95%）、両方のデータを混ぜて97.5%にした。

7. 実装と使用した主な技術

物理シミュレーション MuJoCo 3.17.0、描画 Unreal Engine 5.7.4（Lumen、画面なしのVulkan）、学習 PyTorch 2.10.0、比較用VLA LeRobot 0.4.4 のSmolVLA、指示文の特徴 SmolVLM2-500M-Video-Instruct。アームはAdeept ADA031（5自由度、2本指グリッパー）のシミュレーションモデル。動画は10fpsのシミュレーションを10fpsで書き出した等速で、倍速部分はない（タイトルと終わりの画面は静止画）。

8. 失敗例と考察

・デモの記録を成功判定の瞬間で打ち切ると、グリッパーを開き切る動作が学習データに入らず、方策が「離す直前」で止まった。記録をお手本の最後まで延ばして解決した。

・関節状態を入力に含めた点群VLAは、状態から次の動作を当てる近道を覚え、新芽の位置を読まずに空振りした（15%）。状態入力を外すと点群から位置を読むようになり75%に改善した。

・少数のデモでは、動作全体を丸ごと学ぶより、「どの新芽か・どこか」をAIが推定し、動作は既知の運動学で計画する方が確実（97.5%）。実運用でも、認識と動作計画を分けた方が検証・保守しやすい。

・Unrealのみのデータ（200話）で学習した位置推定は82.5%で、MuJoCoのデータを加えると97.5%に向上した。点群は形と色が主で、画面の質感の差に強い。

9. 制約と今後

新芽は剛体の模型で、切断・柔らかさ・損傷は扱わない。新芽は株の上部にあり遮蔽は少ない。実写茶葉の認識は未実施で、摘む / 残すの判断は指示文の位置指定で代用している。実機への移行には、カメラ校正と実測点群での追加学習が必要。

10. 参考にしたもの・使用した資産

・MuJoCo（Google DeepMind、Apache-2.0）、Unreal Engine 5.7（Epic Games、Unreal Engine EULA）
・UnrealRoboticsLab：MuJoCoとUnreal Engineを連携させる考え方を参考にした（本リポジトリにはコードを含めていない）
・LeRobot と SmolVLA、SmolVLM2（Hugging Face、Apache-2.0）。重みはリポジトリに含めていない
・Adeept ADA031 のシミュレーションモデルと逆運動学：本プロジェクト用に作成し、adeept_sim/ に同梱（キットの説明書の寸法から作成）
・茶樹の参照写真：形と色を合わせるための参考に使った。学習データには使っておらず、リポジトリにも含めない